

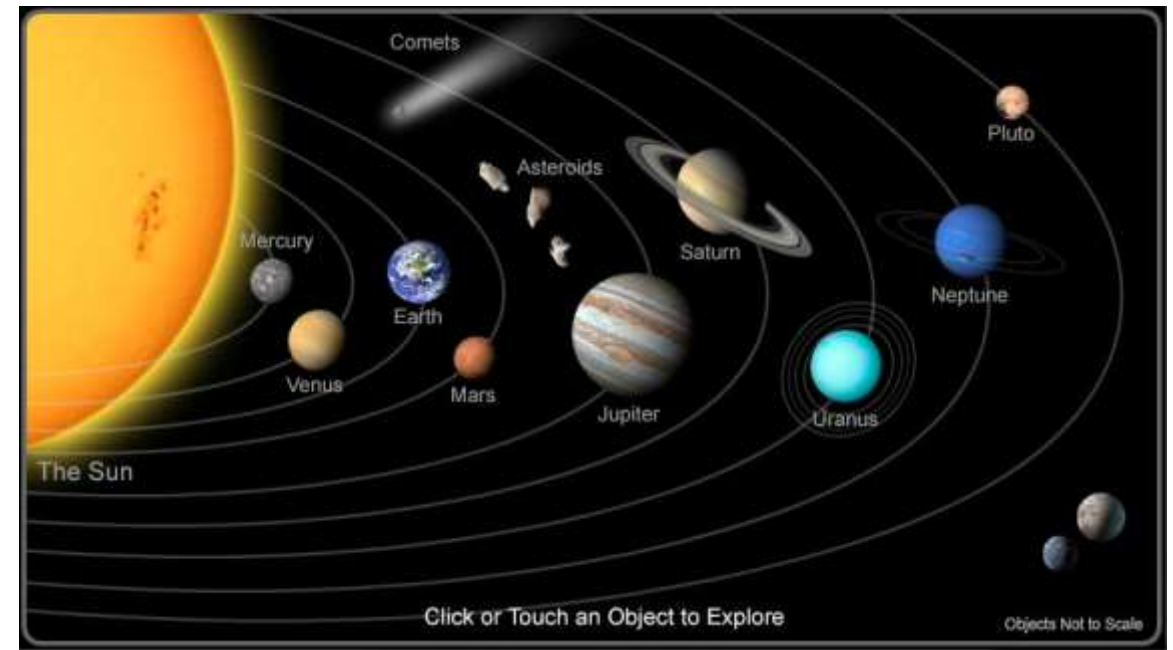
Structures de données avancées

Les graphes : structure et algorithmes

Définition

- Un graphe G est constitué de 2 ensembles V et E .
 - V (Vertices) : un ensemble non vide de sommets.
 - E (Edges) : un ensemble de paires de sommets de V .
 - Ces paires sont appelés aussi des arêtes ou arcs (si orientés)
- On écrit $G = (V, E)$ pour représenter le graphe.
- Les sommets et les arêtes peuvent être étiquetés par informations
- Un graphe est orienté ou non. Dans un graphe non orienté, le parcours d'un sommet à un autre peut se faire dans les deux sens.
- Dans un graphe non orienté, la paire de sommets représentant une arête n'est pas ordonnées. Autrement dit, les (v_1, v_2) et (v_2, v_1) représentent la même arête.

Example



```
43 # (Radius in km, gravity in m/s2)
44 earth = Vertex('earth', (12742, 9807e-3))
45 sun = Vertex('sun', (13927e6, 274))
46
47 # create graph from starting vertices
48 milky_way = EdgeListGraph([earth, sun])
49 # add vertex from provided label and data
50 milky_way.add_vertex('venus', (60518e-1, 887e-2))
51
52 # distance in light years
53 milky_way.add_edge('earth', 'sun', 1581e-8)
54 milky_way.add_edge('earth', 'venus', 44e-7)
55 milky_way.add_edge('sun', 'venus', 11362e-9)
```



Terminologie (1/5)

adjacence	deux arcs sont adjacents s'ils ont une extrémité commune; deux sommets sont adjacents s'il existe un arc, ou une arête, les reliant
arc	couple (x,y) dans un graphe orienté
arête	nom d'un arc, dans un graphe non orienté
boucle	arc reliant un sommet à lui-même
chaîne	Nom d'un chemin dans un graphe non orienté ; séquence d'arcs avec une extrémité commune dans un graphe orienté.
chemin	suite d'arcs connexes reliant un sommet à un autre. Par exemple $(a;b) (b;c) (c;d) (d;b) (b;e)$ est un chemin reliant a à e ; on le note (a,b,c,d,b,e) . Un chemin est une chaîne, la réciproque étant fausse
chemin eulérien	désigne un chemin simple passant une fois et une seule par toutes les arêtes du graphe ; il n'existe pas toujours...
chemin hamiltonien	désigne un chemin simple qui passe une fois et une seule par chaque sommet

Terminologie (2/5)

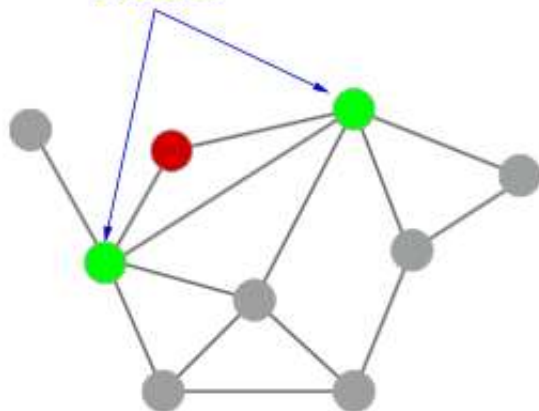
chromatique (nombre)	c'est le nombre minimal de couleurs nécessaires pour colorier tous les sommets du graphe sans que deux sommets adjacents aient la même couleur ; pour les cartes de géographie, le nombre chromatique est 4 ; ce fut, en 1976, la première démonstration informatique d'un théorème mathématiques, sous réserve qu'il n'y ait pas eu de bug dans le programme... il aura fallu 700 pages de listing !
circuit	chemin dont l'origine et l'extrémité sont identiques
Connexité	Un graphe est connexe s'il existe toujours une chaîne, ou un chemin, entre deux sommets quelconques. Par exemple le plan d'une ville doit être connexe.
Complet	un graphe est complet si quels que soient deux sommets distincts, il existe un arc (ou une arête) les reliant dans un sens ou dans l'autre
Cycle	nom d'un circuit dans un graphe non orienté ; dans un graphe orienté, un cycle est une chaîne dont l'extrémité initiale coïncide avec l'extrémité finale
degré d'un sommet	nombre d'arête issues d'un sommet dans un graphe non orienté ; nombre d'arcs arrivant ou partant d'un sommet dans un arc orienté ; on peut vérifier facilement que la somme des degrés de tous les sommets, est donc le double du nombre des arêtes (puisque chacune est comptée deux fois).

Terminologie (3/5)

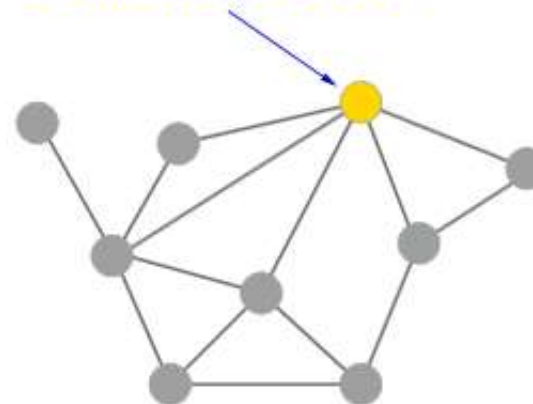
Diamètre	le diamètre d'un graphe est la plus grande chaîne (chemin) de toutes reliant deux sommets quelconques du graphe
Distance	la distance entre deux sommets d'un graphe est la plus petite longueur des chaînes, ou des chemins, reliant ces deux sommets.
graphe orienté	désigne un graphe où le couple (x,y) n'implique pas l'existence du couple (y,x) ; sur le dessin, les liens entre les sommets sont des flèches
graphe simple	désigne un graphe non orienté n'ayant pas de boucle ni plus d'une arête reliant deux sommets. Sur le dessin, les liens entre les sommets sont des segments, et on ne parle alors plus d'arcs mais d'arêtes ; tout graphe orienté peut donc être transformé en graphe simple, en remplaçant les arcs par des arêtes
longueur d'un chemin (ou d'une chaîne)	nombre d'arcs du chemin (ou d'arêtes de la chaîne)
Ordre d'un graphe	nombre de sommets du graphe
prédécesseur	Dans l'arc $(x;y)$, x est prédécesseur de y

Terminologie (4/5)

voisins



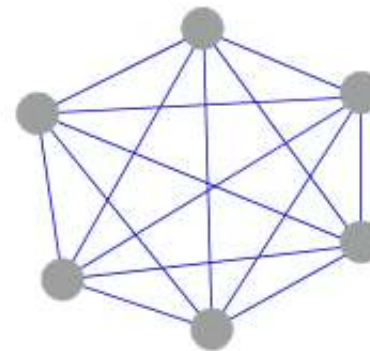
sommet de degré 5



chemin (6 sommets)



graphe complet : $n \times (n-1)/2$ arêtes

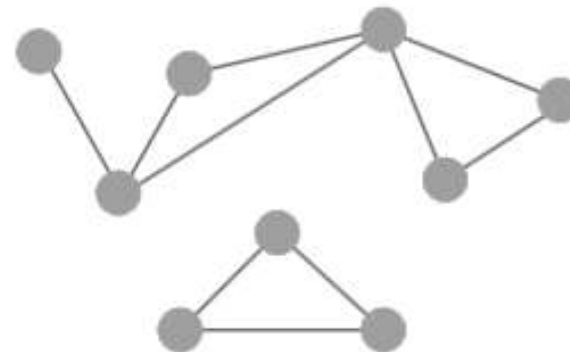


Terminologie (5/5)

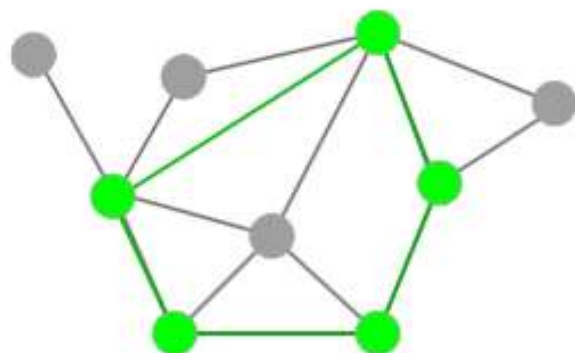
Graphe connexe



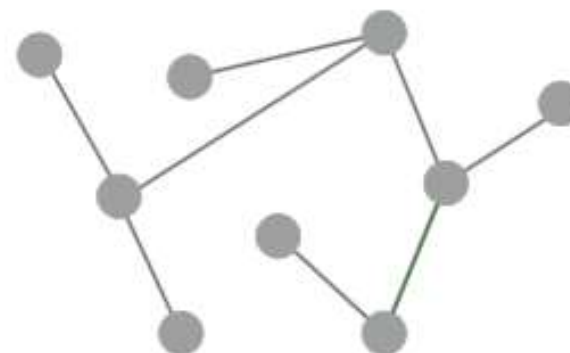
Graphe non connexe



cycle



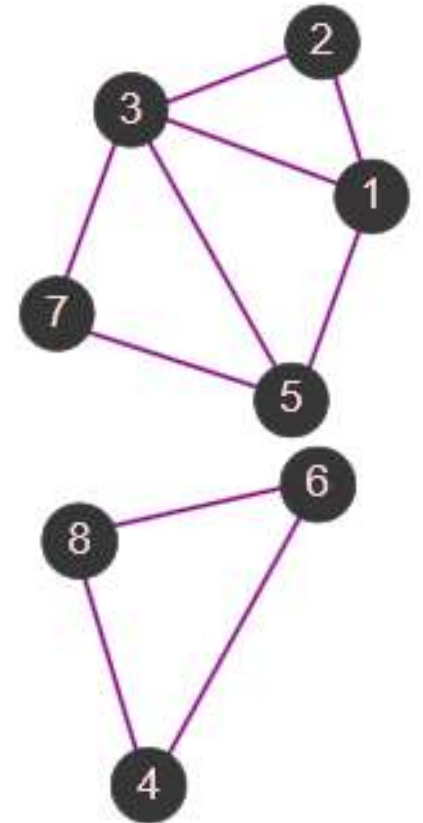
Arbre : connexe, sans cycle



Types de graphes :

Undirected graph

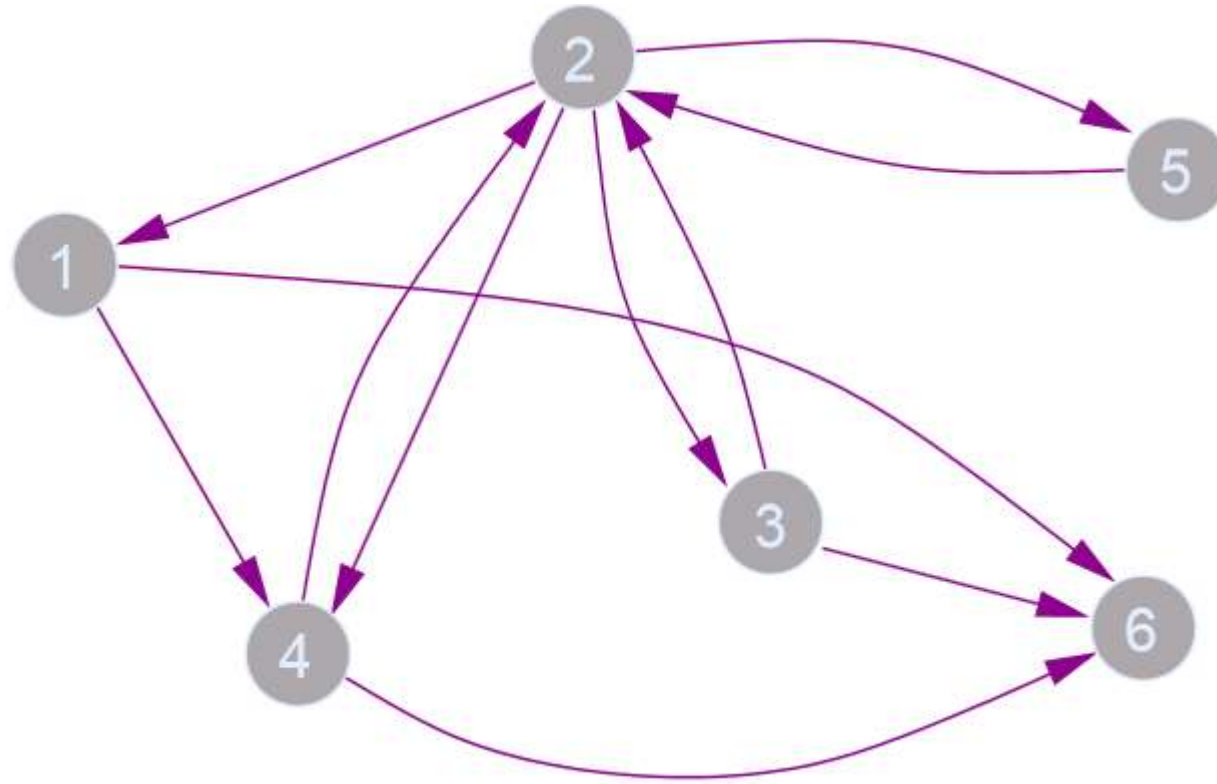
- Un graphe est une paire (V, E) où
 - V (vertices) : un ensemble de nœuds appelés sommets
 - E (edges): un multi-ensemble de paires de sommets appelées arêtes.
 - On peut voir les sommets et les arêtes comme des positions gardant des objets en mémoire
- Exemples:
 - Chaque sommet représente un aéroport et garde en mémoire le code de 3 lettres représentant cet aéroport
 - Chaque arête représente une route aérienne entre deux villes et garde en mémoire la longueur de cette route



Types de graphes :

Directed graph (digraph)

- $G = (V, E)$
- Chaque arête (u, v) est orientée



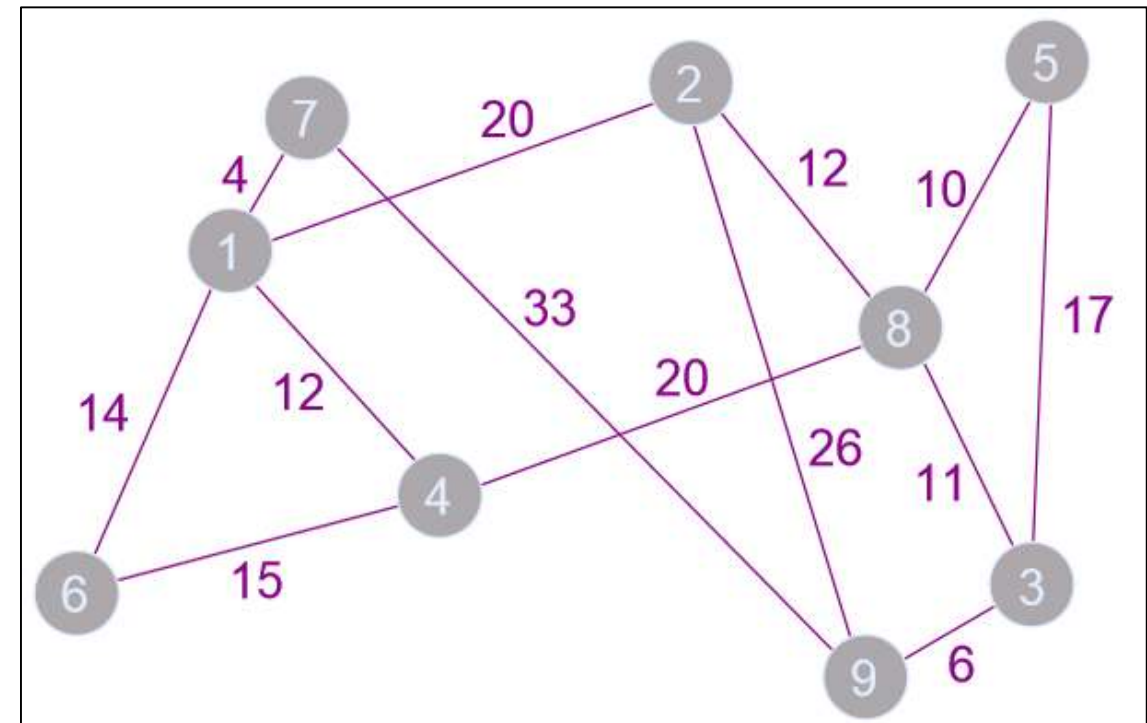
Types de graphes :

Graphe pondéré

- $G = (S, A)$
- Chaque arête (u, v) a un poid $w(u, v)$, aka, coût.

```
28 # distance in light years
29 milky_way.add_edge('sun', 'earth', 1.581E-05)
30 milky_way.add_edge('sun', 'venus', 1.1362E-05)
31 milky_way.add_edge('earth', 'venus', 4.4E-06)
32 milky_way.add_edge('earth', 'jupiter', 8.271665726E-9)
33 milky_way.add_edge('jupiter', 'saturn', 683108E-4)
34 milky_way.add_edge('saturn', 'neptune', 3251757E-3)
35 milky_way.add_edge('jupiter', 'uranus', 2214649E-3)
```

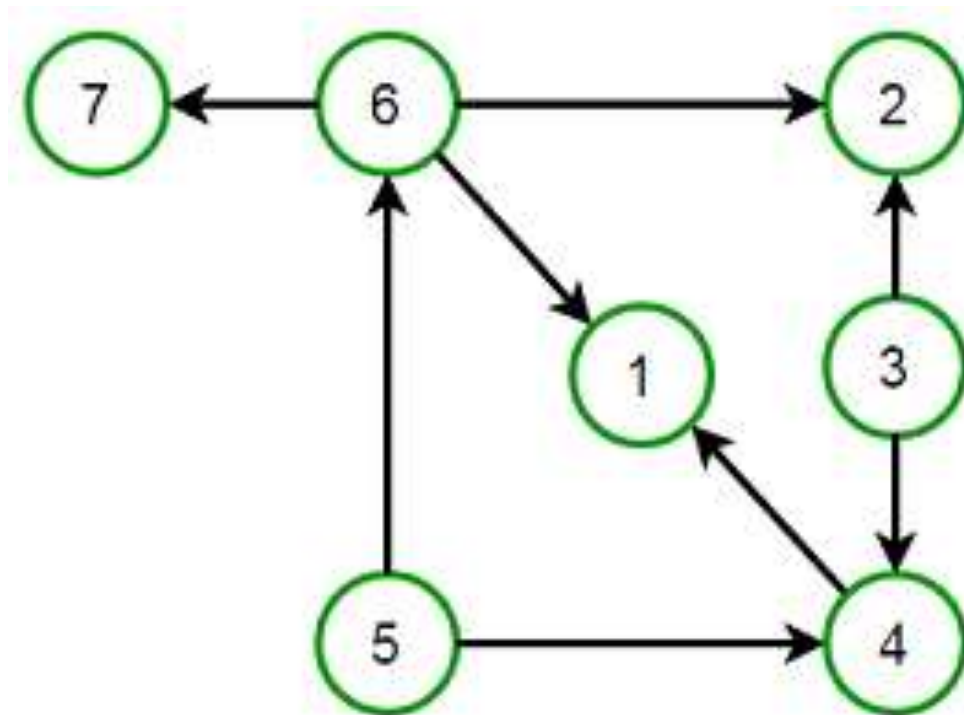
```
11 earth = Vertex('earth', (6371, 9.807))
12 sun = Vertex('sun', (696340, 274))
13 jupiter = Vertex('jupiter', (69911, 24.79))
14 saturn = Vertex('saturn', (58232, 10.44))
15 uranus = Vertex('uranus', (25362, 8.87))
16 pluto = Vertex('pluto', (1188.3, 0.62))
```



Types de graphes :

Graphe acyclique (Directed Acyclic Graph –DAG –)

- Un graphe orienté qui ne contient pas de cycle.



TAD Graphe

```
3 class IGraph(object):
4
5     # Assuming vertex v is one endpoint of the edge
6     # (either origin or destination), return the other endpoint.
7     def _connections(self, vertex):
8         pass
9
10    # Return the number of vertices of the graph.
11    def _vertex_count(self):
12        pass
13
14    # Return the number of edges of the graph.
15    def _edge_count(self):
16        pass
17
18    # Return an iteration of all the vertices of the graph.
19    def _vertices(self):
20        pass
```

```
22    # Return an iteration of all the edges of the graph.
23    def _edges(self):
24        pass
25
26    # Return the edge from vertex u to vertex v, if one exists;
27    # otherwise return None. For an undirected graph, there is
28    # no difference between get edge(u,v) and get edge(v,u).
29    def _get_edge(self, u, v):
30        pass
31
32    # For an undirected graph, return the number of edges incident
33    # to vertex v. For a directed graph, return the number
34    # of outgoing (resp. incoming) edges incident to vertex v,
35    # as designated by the optional parameter.
36    def _degree(self, v):
37        pass
```

```
39    # Create and return a new Vertex storing element data.
40    def _add_vertex(self, key, data=None):
41        pass
42
43    # Create and return a new Edge from vertex u to vertex v,
44    # storing element data (None by default).
45    def _add_edge(self, u, v, data=None):
46        pass
47
48    # Remove vertex v and all its incident edges from the graph.
49    def _remove_vertex(self, key):
50        pass
51
52    # Remove edge e from the graph.
53    def _remove_edge(self, e):
54        pass
55
56    def is_directed(self):
57        pass
```

Implémenter un graphe :

Représenter les sommets et les arêtes

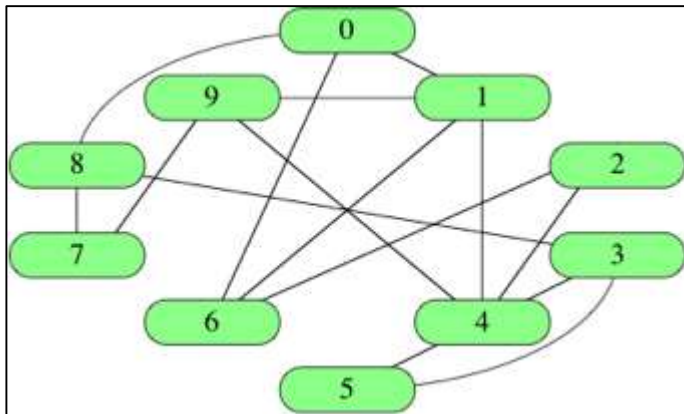
```
2 class Vertex(object):
3
4     def __init__(self, label, data):
5         self.key = label
6         self.data = data
7         # self.__neighbors = {}
8
9     @property
10    def label(self):
11        return self.key
12
13    def hash(self):
14        return hash(self.__key) # hash(id(self))
15
16    def __str__(self):
17        return f'[{str(self.key)}, {str(self.data)}']
```

```
2 class Edge(object):
3     def __init__(self, source_vertex, destination_vertex, data):
4         self.origin = source_vertex
5         self.destination = destination_vertex
6         self.data = data
7
8     def hash(self):
9         return hash((self.destination, self.destination))
10
11    def __str__(self):
12        return f'\n({str(self.origin)}, {str(self.destination)}, {str(self.data)})'
```

Implémenter un graphe :

Liste d'arêtes – Le principe

- Le graphe est une collection de triplets:
 - Sommet source
 - Sommet destination
 - Les informations de l'arête (ex. le poids)
- Le graphe doit aussi avoir une liste de sommets car le graphe peut avoir des sommets sans arêtes (non connectés).
- Structure simple mais opérations complexes.



```
[ [0,1], [0,6], [0,8], [1,4],  
  [1,6], [1,9], [2,4], [2,6],  
  [3,4], [3,5],  
  [3,8], [4,5], [4,9], [7,8], [7,9] ]
```

Implémenter un graphe : Liste d'arêtes – Le code

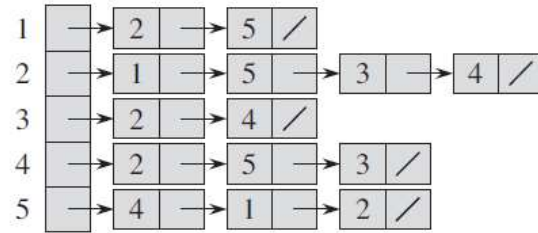
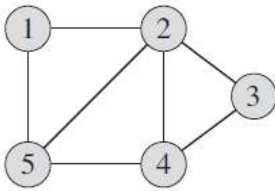
```
5 class EdgeListGraph(object):
6
7     # Graph = (V, E)
8     def __init__(self, vertices=[], edges=[],
9                 directed=False):
10
11         # vertices are mandatory for a
12         # non-connected vertex
13         self.vertices = vertices
14         self.edges = edges
15         self.directed = directed
16
17     def add_vertex(self, key, data):
18         new_vertex = Vertex(key, data)
19         self.vertices.append(new_vertex)
20
21     def add_edge(self, new_edge):
22         if new_edge is not None:
23             self.edges.append(new_edge)
24
25     def remove_edge(self, edge):
26         self.edges.remove(edge)
```

```
25     def add_edge(self, origin_key, destination_key, data):
26         origin_vertex = self.find_vertex(origin_key)
27         destination_vertex = self.find_vertex(destination_key)
28         new_edge = Edge(origin_vertex, destination_vertex, data)
29         self.edges.append(new_edge)
30
31     def find_vertex(self, key):
32         for vertex in self.vertices:
33             if vertex.key == key:
34                 return vertex
35         return None
36
37     def __str__(self):
38         return ''.join(map(str, self.edges))
```

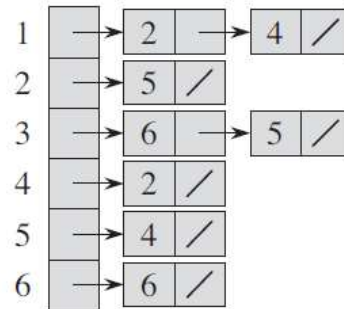
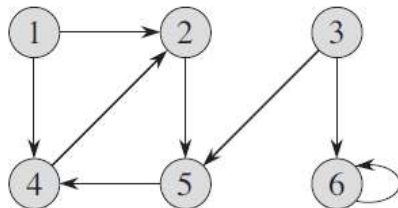
Implémenter un graphe: les listes d'adjacences – Le principe

- Une table de hachage pour stocker les sommets
- À chaque sommet est associé une liste de voisins
- Les graphes orientés enregistrent seulement les sommets au bout des arcs sortants.

Graphe non dirigé



Graphe dirigé



- Complexité en espace : $O(|V| + |E|)$
 - ▶ optimal
- Accéder à un sommet : $O(1)$
 - ▶ optimal
- Parcourir tous les sommets : $\Theta(|V|)$
 - ▶ optimal
- Parcourir toutes les arêtes : $\Theta(|V| + |E|)$
 - ▶ ok (mais pas optimal)
- Vérifier l'existence d'une arête $(u, v) \in E$: $O(|V|)$
 - ▶ ou encore $O(\min(\text{degree}(u), \text{degree}(v)))$
 - ▶ mauvais

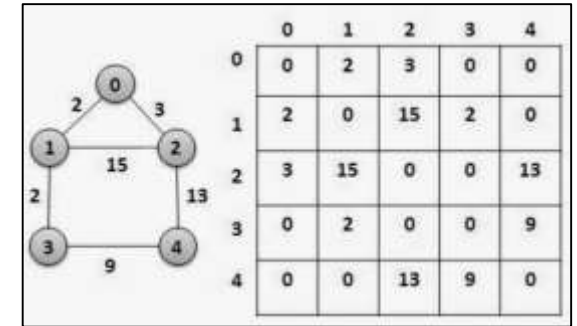
Implémenter un graphe: les listes d'adjacences – Le code

```
5 class AdjacencyListGraph(object):
6
7     def __init__(self, vertices=[], directed=True):
8         # hashtable that maps vertex to a list of edges
9         # key is vertex, value is connected edges
10        # dict.fromkeys(vertices, None)
11        self.graph_dict = {key: [] for key in vertices}
12        self.directed = directed
13
14        # Return an iteration of all vertices of the graph
15    def vertices(self):
16        return self.graph_dict.keys()
17
18        # Return the number of edges in the graph
19    def edge_count(self):
20        total = sum(len(self.graph_dict[v]) for v in self.graph_dict)
21        return total
```

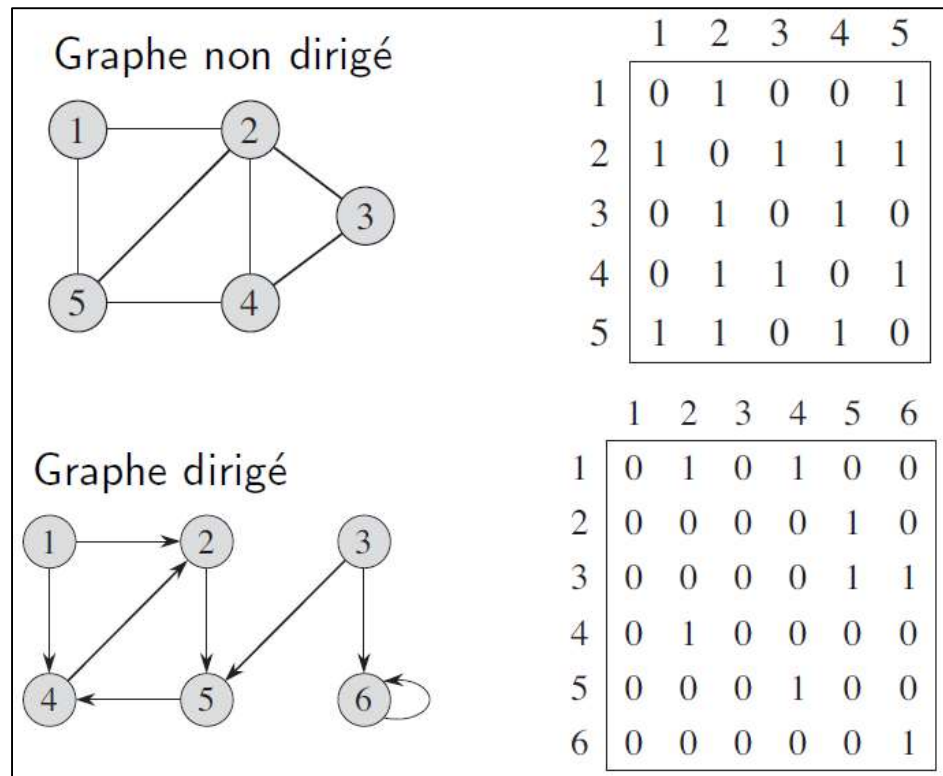
```
30     def find_vertex(self, key):
31         for vertex in self.graph_dict.keys():
32             if vertex.key == key:
33                 return vertex
34         return None
35
36     def add_vertex(self, key, data):
37         new_vertex = Vertex(key, data)
38         self.graph_dict[new_vertex] = []
39
40     def add_edge(self, origin_key, destination_key, data=None):
41         origin_vertex = self.find_vertex(origin_key)
42         destination_vertex = self.find_vertex(destination_key)
43         new_edge = Edge(origin_vertex, destination_vertex, data)
44         self.graph_dict[origin_vertex].append(new_edge)
```

Implémenter un graphe:

Matrice d'adjacences – Le principe



- Étant donné un graphe $G=(V,E)$ avec $V=\{1, \dots, n\}$
- La matrice $A=(a_{ij})$ de dimension $n \times n$ avec $a_{ij}=1$ si (i,j) dans E (ou le poids w_{ij}) ou $a_{ij}=0$ sinon (ou le poids infini)



- Complexité en espace : $O(|V|^2)$
 - potentiellement très mauvais
- Accéder à un sommet : $O(1)$
 - optimal
- Parcourir tous les sommets : $\Theta(|V|)$
 - optimal
- Parcourir toutes les arêtes : $\Theta(|V|^2)$
 - potentiellement très mauvais
- Vérifier l'existence d'une arête $(u, v) \in E$: $O(1)$
 - optimal

Implémenter un graphe:

Matrice d'adjacences – Le code

```
7      # Graph = (V, E)
8      def __init__(self, vertices=[], edges=[], directed=True, max_size=10):
9          self.vertices = dict()
10         self.size = 0
11         self.max_size = max_size
12         for vertex in vertices:
13             self.vertices[vertex.key] = (vertex, self.size)
14             self.size += 1
15         self.matrix = [[None] * self.max_size for _ in range(self.max_size)]
16         self.directed = directed
```

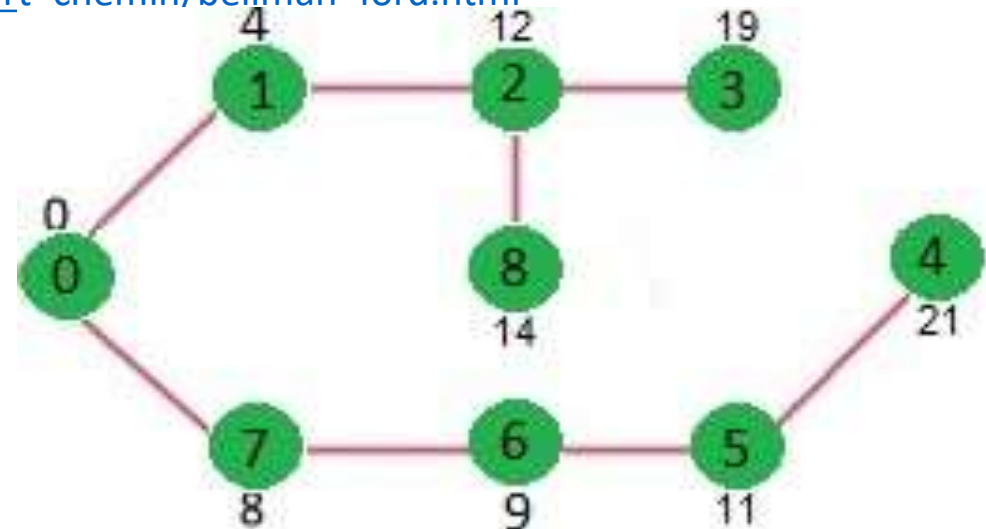
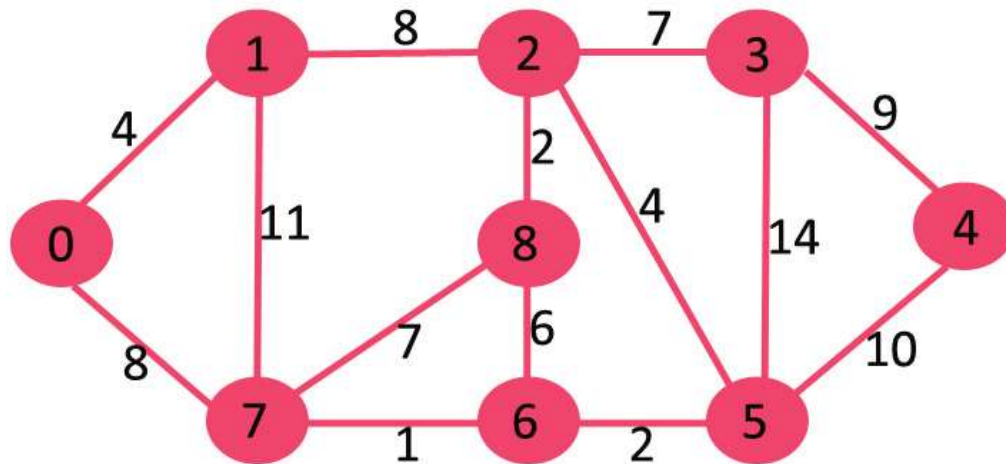
```
18     def find_index(self, vertex_key):
19         if vertex_key in self.vertices:
20             return self.vertices[vertex_key][1]
21         else:
22             return -1
23
24     def find_vertex(self, vertex_key):
25         if vertex_key in self.vertices:
26             return self.vertices[vertex_key][0]
27         else:
28             return None
```

```
34     def add_vertex(self, key, data):
35         new_vertex = Vertex(key, data)
36         self.vertices[key] = (new_vertex, self.size)
37         self.size += 1
38
39     def add_edge(self, source_key, destination_key, data):
40         u = self.find_index(source_key)
41         v = self.find_index(destination_key)
42         self.matrix[u][v] = data
43         # if self.directed:
44         #     self.matrix[v][u] = data
```

Algorithmes de graphes populaires :

Le plus court chemin dans un graphe

- Graphe orienté (digraph) ou non.
- Calcul des plus courts chemins à partir d'un sommet donné vers un autre sommet
- Plusieurs algorithmes
 - Dijkstra (poids positifs uniquement)
 - https://haltode.fr/algo/structure/graphe/plus_court_chemin/dijkstra.html
 - Bellman-Ford (poids négatifs aussi)
 - https://haltode.fr/algo/structure/graphe/plus_court_chemin/bellman_ford.html



Domaines d'applications des graphes

- Réseaux sociaux
- Réseaux informatiques
- Télécommunications
- World Wide Web
- Cartes routières
- Etc.

Références

- Introduction to algorithms, <http://courses.csail.mit.edu/6.006/fall10/>
- <http://www.dil.univ-mrs.fr/~gcolas/algo-licence/slides/graphes-algo1.pdf>
- <https://www.baeldung.com/java-graphs>
- <https://opendatastructures.org/versions/edition-0.1d/ods-java/node59.html>
- <https://www.youtube.com/watch?v=85bHg5AipvU&list=PLLMXbkbDbVt8JLumqKj-3BlHmEXPIfR42>
- <https://www.geeksforgeeks.org/graph-data-structure-and-algorithms/>
- <https://algorithms.tutorialhorizon.com/site-map/>
- http://www.btechsmartclass.com/data_structures/graph-traversal-bfs.html